WEEK 4 · DAY 4 · OVERVIEW

From Notebook to Module



Created by **Jeff Lenamon**

ABOUT 40 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

The EDA notebook's cleaning routine lives in one notebook, and October's extract arrives next month. Put it and its checks in one file anyone on the desk can import. Run it on September and tie the cover note. Count what it changes, and prove it leaves a file with no problems alone. Write me the workbook. Test the function itself, so an edit next year cannot break it quietly. And tell me which of our other spreadsheet jobs deserve the same.

Your Route Through Day 4

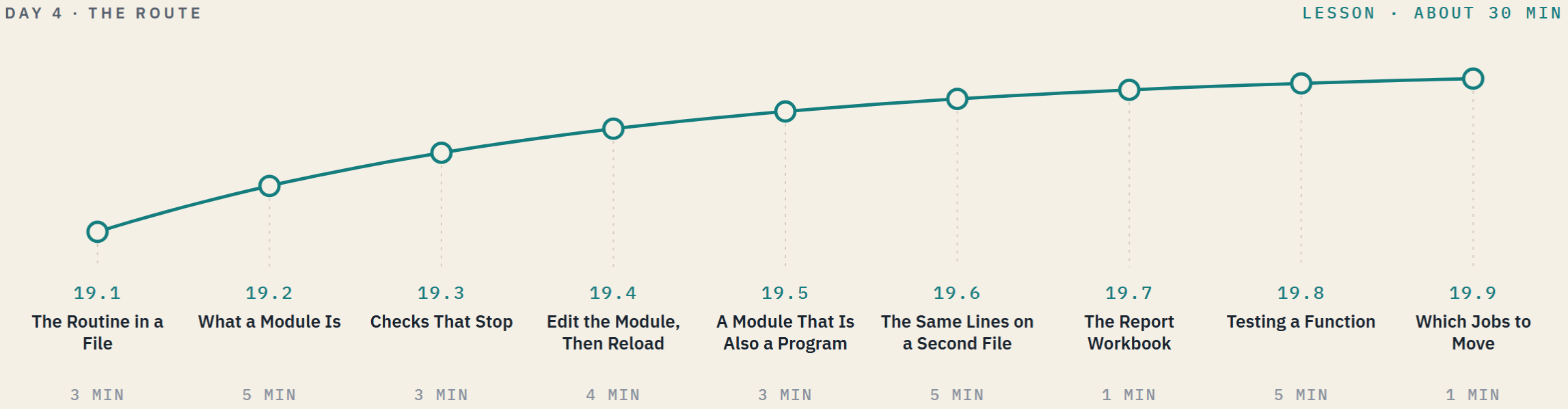


FIGURE 1 Nine sections, in order, with about how long each takes.

Time. About 30 minutes for the lesson, then about 12 for the four exercises.

19.1 The Routine in a File

desk_tools.py: one file, five functions

```
"""Desk tools: load, clean, check, summarize, and report..."""     import pandas as pd
```

function	what goes in	what comes out
`load_holdings(path)`	a CSV file name or web address	the extract as it arrived
`clean_holdings(raw)`	the extract	the cleaned book; raw unchanged
`check_totals(book, bonds, face, market_value)`	a table, three totals in dollars	nothing, or a ValueError
`summarize(book)`	a cleaned book	one row: dollars, bp, percent
`write_report(book, summary, path)`	the book, its summary, a file name	a workbook, two sheets

FIGURE 2 desk_tools.py as one page: a docstring, its import of pandas, and five functions, each with what goes in and out.

Run September From the File

WRITE THE MODULE

```
def load_holdings(path: str) -> pd.DataFrame:
def clean_holdings(raw):
def check_totals(book, bonds, face, market_value):
def write_report(book, summary, path):
```

IMPORT IT AND RUN SEPTEMBER

```
import desk_tools
raw = desk_tools.load_holdings(data_folder + 'holdings_messy.csv')
book = desk_tools.clean_holdings(raw)
desk_tools.check_totals(book, expected_bonds, expected_face, expected_value)
print(f"{len(raw)} rows in, {len(book)} out, {book['market_value'].sum():,.2f} dollars")
```

```
204 rows in, 200 out, 490,819,215.00 dollars
```

Five lines ran the EDA notebook's routine from a file and tied the cover note to the cent.

19.2 What a Module Is

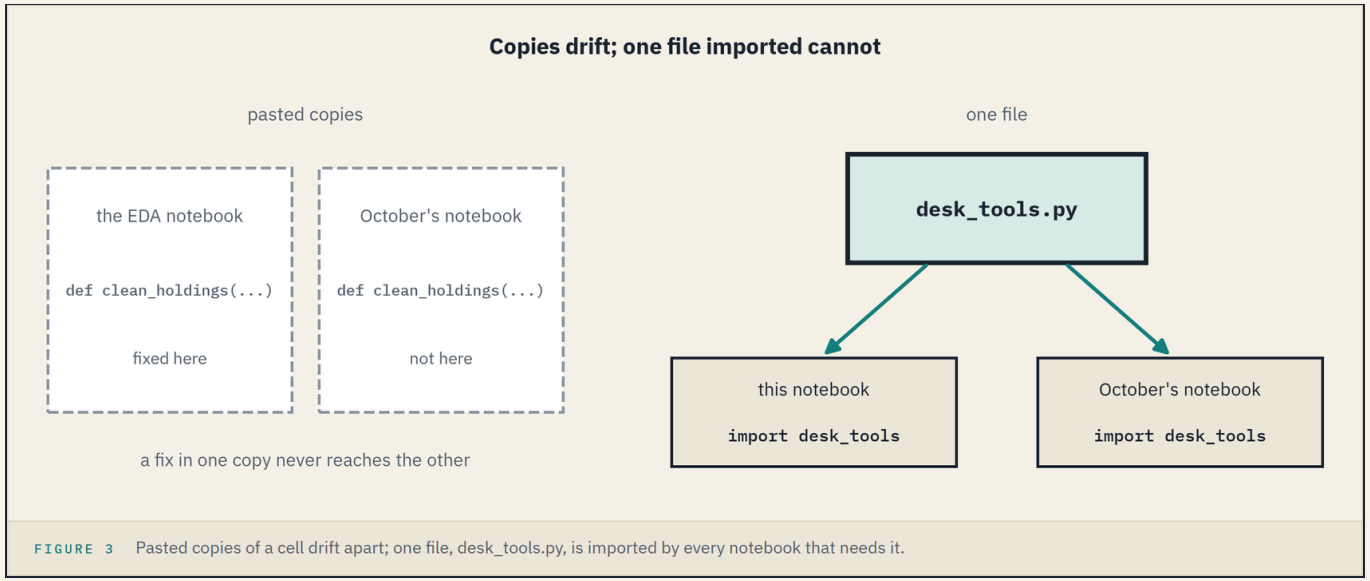


FIGURE 3 Pasted copies of a cell drift apart; one file, desk_tools.py, is imported by every notebook that needs it.

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Steps every workbook can use	`import desk_tools`	macros in the Personal Macro Workbook, PERSONAL.XLSB, which Excel opens hidden at start-up
A named function you write	`def`	`=LAMBDA(oas, oas / 100)` named in Formulas, Name Manager, for that workbook only (Microsoft 365, Excel 2024)
A function's arguments	`help()`	type `=SUM` in a cell, then Ctrl+A: the Function Arguments box

The file documents itself: what goes in, what comes out, in which unit.

19.3 Checks That Stop

CLEANING LEAVES THE EXTRACT ALONE

```
untidy = raw['ticker'] != raw['ticker'].str.strip().str.upper()
print(len(raw), raw.duplicated().sum(), untidy.sum(), raw['rating'].isna().sum())
```

```
204 4 8 5
```

THE CLEANED BOOK HEADLINE

BONDS	FACE	MARKET_VALUE	MEAN_OAS
200	489,000,000	490,819,215	185.75

EXCEL SIDE BY SIDE

CHECK	EXCEL
Bonds	`=COUNTA(B2:B201)`
Face	`=SUM(R2:R201)`
Market value	`=SUM(S2:S201)`
One is off	the EDA notebook's `IF` cell only shows CHECK; `check_totals` stops the run

The routine stops on a file that does not tie, and never changes the one it was given.

19.4 Edit the Module, Then Reload

APPEND A SIXTH FUNCTION

```
def count_changes(raw, book):
    counts = {
        'rows_removed': len(raw) - len(book),
        'tickers_tidied': (kept['ticker'] != book['ticker']).sum(),
        'stale_prices_flagged': book['stale_price'].sum(),
        'ratings_labelled': kept['rating'].isna().sum(),
        'yields_corrected': (kept['yield_pct'] != book['yield_pct']).sum(),
    }
    return pd.DataFrame([counts])
```

RELOAD IT

```
import importlib
importlib.reload(desk_tools)
desk_tools.count_changes(raw, book)
```

ONE COUNT FOR EACH DECISION

	ROWS_REMOVED	TICKERS_TIDIED	STALE_PRICES_FLAGGED	RATINGS_LABELLED	YIELDS_CORRECTED
0	4	8	6	5	2

After every edit: reload, or restart and run from the top.

19.5 A Module That Is Also a Program

THE BLOCK AT THE END OF THE FILE

```
%%writefile -a desk_tools.py
if __name__ == '__main__':
    print('desk_tools holds the month-end functions: import it to use them.')
```

RUN IT AS A PROGRAM

```
# %run, as in Day 0, runs a .py file as a program
%run desk_tools.py
```

```
desk_tools holds the month-end functions: import it to use them.
```

Above the line, a module to import; below it, what the file does when run.

19.6 The Same Lines on a Second File

BOTH FILES THROUGH THE ROUTINE

```
for file in files:
    received = desk_tools.load_holdings(data_folder + file)
    books[file] = desk_tools.clean_holdings(received)
    desk_tools.check_totals(books[file], expected_bonds, expected_face, expected_value)
    summaries[file] = desk_tools.summarize(books[file])
    changes[file] = desk_tools.count_changes(received, books[file]).iloc[0]
```

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Next month's file	change the file name	re-point every range: `S2:S205` on the 204-row extract, `S2:S201` on the 200-row file
Same steps, new source	the loop above	Power Query: Data, Get Data, Data Source Settings, Change Source, then Refresh (Excel 2016 and later)

One file name changed, and the clean file came through untouched.

19.7 The Report Workbook

WRITE IT, READ IT BACK

```
report_file = 'month_end_report.xlsx'
desk_tools.write_report(book, summary, report_file)
summary_back = pd.read_excel(report_file, sheet_name='summary')
book_back = pd.read_excel(report_file, sheet_name='book', dtype={'cusip': str})
print(summary_back.shape, book_back.shape)
```

```
(1, 7) (200, 20)
```

EXCEL SIDE BY SIDE

JOB	PANDAS	EXCEL
Two tables, one file	`pd.ExcelWriter`, one `to_excel(sheet_name=)` each	one workbook, two sheets
Totals of the sheet	`check_totals(book_back, ...)`	`=COUNTA(book!B2:B201)`, `=SUM(book!R2:R201)`, `=SUM(book!S2:S201)`

The PM's workbook still ties after the round trip.

19.8 Testing a Function

Seven rows, worked by hand: what clean_holdings should return

tiny: one of each problem

row	cusip	ticker	rating	yield	price_date
0	TEST00001	KRIX	BBB	5.1	2026-09-30
1	TEST00002	' krix'	BBB	6.2	2026-09-30
2	TEST00003	MORX	NaN	5.5	2026-09-30
3	TEST00004	MORX	A	4.9	2026-08-14
4	TEST00005	THAA	BB	0.0575	2026-09-30
5	TEST00006	CORA	B	5.8	2026-09-30
6	TEST00006	CORA	B	5.8	2026-09-30

row 6 repeats row 5

expected: six rows, each fix lit

row	cusip	ticker	rating	yield	stale
0	TEST00001	KRIX	BBB	5.1	False
1	TEST00002	KRIX	BBB	6.2	False
2	TEST00003	MORX	Not rated	5.5	False
3	TEST00004	MORX	A	4.9	True
4	TEST00005	THAA	BB	5.75	False
5	TEST00006	CORA	B	5.8	False

the repeat is gone

FIGURE 6 The seven rows of tiny with each problem lit, beside the six rows expected, worked by hand.

Tests You Can Run in One Line

EVERY TEST IN ONE FUNCTION

```
def test_clean_holdings():
    result = desk_tools.clean_holdings(tiny)
    assert len(result) == 6, f'expected 6 rows, found {len(result)}'
    assert result['ticker'].tolist() == expected_tickers, 'tickers not tidied'
    assert result.loc[2, 'rating'] == 'Not rated', 'missing rating not labelled'
    assert result['yield_pct'].tolist() == [5.1, 6.2, 5.5, 4.9, 5.75, 5.8], 'yields wrong'
    assert result['stale_price'].tolist() == [False, False, False, True, False, False], 'stale'
```

```
clean_holdings: 5 checks passed
```

A TEST THAT FAILS

```
assert tidy.loc[2, 'rating'] == 'NR', f"expected the label NR, found {tidy.loc[2, 'rating']}"
```

```
AssertionError: expected the label NR, found Not rated
```

A test fails loudly where the totals stay silent.

19.9 Which Jobs to Move

THE SIGNS

SIGN	SEPTEMBER'S ROUTINE
It repeats	every month end
The input changes shape	204 rows, then 200
It has control totals	the cover note's three
Others rely on it	the PM's workbook
The steps are hard to see	six decisions, now in one docstring

Move a job when it repeats, changes shape, has control totals, feeds others, and hides its steps.

You Can Now

- Write a module with **`%%writefile`**, and append with **`%%writefile -a`**.
- Import it, and read it with **`dir()`** and **`help()`**.
- Reload it after an edit, and say why.
- Read **`if __name__ == '__main__':`**, and run a file with **`%run`**.
- Run the same lines on a second file, and count each decision's changes.
- Write a two-sheet workbook, read it back, and check it.
- Test a function on a table worked by hand.
- Say which jobs belong in a module.

NEXT

WEEK 4 · DAY 5

Course Project

A new month-end extract is tested against its cover note, cleaned by a function you write, described, compared with its benchmark, and written up for the portfolio manager. You write the code.